

Projet Pokédex

Capter des Pokémon depuis une API publique et les stocker dans Neon

Cours : **Service Web (25604)** • Travail pratique

Stack : Node.js, TypeScript, Express 5, Prisma 7, PostgreSQL (Neon), **PokéAPI**

Le scénario. Vous venez de recevoir votre tout premier Pokédex ! Mais il est **vide**. Votre mission de Dresseur · euse : interroger le grand serveur mondial des Pokémon (la **PokéAPI**), afficher les créatures disponibles dans votre backend, puis **capturer** celles que vous voulez pour les enregistrer pour toujours dans votre base de données sur Neon. Chaque quête réussie rapporte des points d'expérience (**XP**). Devenez **Maître Pokémon** en complétant tout le Pokédex de l'exercice !

Objectifs pédagogiques

- Consommer une **API REST externe** depuis un backend Node (avec `fetch`).
- **Transformer** les données reçues (mapping de champs, conversion d'unités, enums).
- Modéliser ces données avec Prisma (enums + relation 1-N) et les **persister** sur PostgreSQL/-Neon.
- Construire des routes Express qui combinent *appel API* et *écriture en base*.

1. Quête 1 — Préparer le sac à dos (installation)

[100 XP]

Nouveau dossier `pokedex-api`. Ouvrez-y un terminal PowerShell.

À faire

Initialiser le projet et installer les dépendances.

Listing 1 – Initialisation & dépendances

```
1 npm init -y
2
3 # Production
4 npm install express dotenv @prisma/client @prisma/adaptor-neon
5
6 # Développement
7 npm install --save-dev typescript tsx @types/node @types/express prisma
8
9 npx tsc --init
```

Bonne nouvelle : pas besoin d'installer `axios` ni `node-fetch`. À partir de **Node 18**, la fonction `fetch` est **native**. Vérifiez votre version avec `node -v` (il faut ≥ 18).

Scripts à ajouter dans `package.json` :

Listing 2 – package.json (extrait)

```
1 "scripts": {  
2   "dev": "tsx watch src/server.ts"  
3 }
```

2. Quête 2 — Le Centre Pokémon (Neon + .env)

[100 XP]

À faire

Créez une base PostgreSQL gratuite sur neon.tech, puis un fichier `.env` à la racine :

Listing 3 – .env

```
1 DATABASE_URL="postgresql://utilisateur:motdepasse@hote.neon.tech/nomdb?sslmode=  
   require"  
2 PORT=3000
```

Ajoutez un `.gitignore` avec `node_modules/`, `.env` et `generated/`. Ne committez **jamais** le `.env`.

3. Quête 3 — Le Pokédex national (schéma Prisma)

[250 XP]

On modélise un **Dresseur** qui possède des **Pokémon** (relation 1–N), avec des **enums** pour le type et la rareté.

À faire

1. `npx prisma init`.
2. Remplacez `prisma/schema.prisma` par le schéma ci-dessous.

Listing 4 – prisma/schema.prisma

```
1 generator client {  
2   provider = "prisma-client"  
3   output   = "../generated/prisma"  
4 }  
5  
6 datasource db {  
7   provider = "postgresql"  
8 }  
9  
10 // ----- ENUMS -----  
11 enum TypePokemon {  
12   NORMAL FEU EAU PLANTE ELECTRIK GLACE  
13   COMBAT POISON SOL VOL PSY INSECTE  
14   ROCHE SPECTRE DRAGON TENEBRES ACIER FEE  
15 }  
16  
17 enum Rarete {  
18   COMMUN  
19   RARE  
20   LEGENDAIRE  
21 }  
22
```

```

23 // ----- MODELES -----
24 model Dresseur {
25   id          Int          @id @default(autoincrement())
26   pseudo      String       @unique
27   ville       String?
28   badges      Int          @default(0)
29   createdAt   DateTime     @default(now())
30   pokemons   Pokemon[]
31 }
32
33 model Pokemon {
34   id          Int          @id @default(autoincrement())
35   numeroPokedex Int        @unique           // l'id provenant de la PokeAPI
36   nom         String
37   typePrincipal TypePokemon
38   typeSecondaire TypePokemon?           // certains Pokemon ont 2 types
39   pointsVie   Int
40   taille      Float         // en metres
41   poids       Float         // en kilogrammes
42   imageUrl    String?
43   rarete      Rarete        @default(COMMUN)
44   captureLe   DateTime     @default(now())
45   // Relation 1-N : un Pokemon appartient (optionnellement) a un dresseur
46   dresseurId  Int?
47   dresseur    Dresseur?     @relation(fields: [dresseurId], references: [id])
48 }

```

numeroPokedex est @unique : on ne capture chaque espèce qu'une seule fois. Les enums garantissent que typePrincipal ne prendra qu'une valeur valide. Le ? rend un champ optionnel (un Pokémon peut n'avoir qu'un seul type).

4. Quête 4 — La Poké Ball de connexion (utils/prisma.ts)

[100 XP]

Créez `utils/prisma.ts` avec **exactement** ce contenu (pattern standard du cours : adaptateur Neon + logs) :

Listing 5 – `utils/prisma.ts`

```

1 import { PrismaNeon } from "@prisma/adaptor-neon"
2 import { PrismaClient } from "../generated/prisma/client"
3 import dotenv from "dotenv"
4 dotenv.config();
5
6 const adapter = new PrismaNeon({
7   connectionString : process.env.DATABASE_URL,
8 })
9
10 const prisma = new PrismaClient({
11   adapter,
12   log: ['query', 'info', 'warn', 'error'],
13 })
14
15 export default prisma;

```

L'import du client vient de `../generated/prisma/client` (le dossier output du schéma), **pas** de `@prisma/client`.

5. Quête 5 — Réveiller les tables (migration)

[150 XP]

```
1 npx prisma migrate dev --name init
2 npx prisma generate
```

À relancer après chaque modification du schéma. Si Neon répond « Can't reach database server », votre base s'est mise en pause : ouvrez le dashboard Neon pour la réveiller.

6. Quête 6 — Se connecter à la PokéAPI

[300 XP]

La **PokéAPI** est **publique et gratuite** (aucune clé requise). Deux points d'entrée nous intéressent :

URL	Renvoi
.../api/v2/pokemon?limit=20	une <i>liste</i> { name, url }
.../api/v2/pokemon/pikachu	le <i>détail</i> d'un Pokémon

Le détail contient des champs à **transformer** : **height** est en *décimètres*, **weight** en *hectogrammes* (on divise par 10), les **types** sont en *anglais minuscule* (à mapper vers notre enum), et les PV sont cachés dans le tableau **stats**.

À faire

Créez `src/routes/pokedex.routes.ts` avec une fonction utilitaire `recupererPokemon` et deux routes de *lecture* de l'API.

Listing 6 – `src/routes/pokedex.routes.ts` (partie 1 : lire l'API)

```
1 import { Router, Request, Response } from "express"
2 import prisma from "../../utils/prisma"
3
4 const routerPokedex = Router()
5 const POKEAPI = "https://pokeapi.co/api/v2"
6
7 // Mapping des types anglais (PokeAPI) -> notre enum français
8 const TYPES: Record<string, string> = {
9   normal: "NORMAL", fire: "FEU", water: "EAU", grass: "PLANTE",
10  electric: "ELECTRIK", ice: "GLACE", fighting: "COMBAT", poison: "POISON",
11  ground: "SOL", flying: "VOL", psychic: "PSY", bug: "INSECTE",
12  rock: "ROCHE", ghost: "SPECTRE", dragon: "DRAGON", dark: "TENEbres",
13  steel: "ACIER", fairy: "FEE",
14 }
15
16 // Recupere UN pokemon depuis la PokeAPI et le transforme pour notre base
17 async function recupererPokemon(nom: string) {
18   const reponse = await fetch(`${POKEAPI}/pokemon/${nom.toLowerCase()}`)
19   if (!reponse.ok) return null
20   const data: any = await reponse.json()
21
22   const hp = data.stats.find((s: any) => s.stat.name === "hp")?.base_stat ?? 0
23
24   return {
25     numeroPokedex: data.id,
26     nom: data.name,
27     typePrincipal: TYPES[data.types[0].type.name] ?? "NORMAL",
28     typeSecondaire: data.types[1] ? TYPES[data.types[1].type.name] : null,
```

```

29     pointsVie: hp,
30     taille: data.height / 10,    // decimetres -> metres
31     poids: data.weight / 10,    // hectogrammes -> kg
32     imageUrl: data.sprites.front_default,
33   }
34 }
35
36 // GET : liste des Pokemon disponibles dans l'API -> /pokedex/api?limite=20
37 routerPokedex.get("/api", async (req: Request, res: Response) => {
38   const limite = Number(req.query.limite) || 20
39   const reponse = await fetch(`${POKEAPI}/pokemon?limit=${limite}`)
40   const data: any = await reponse.json()
41   res.json(data.results) // [{ name, url }, ...]
42 })
43
44 // GET : detail d'un Pokemon depuis l'API -> /pokedex/api/pikachu
45 routerPokedex.get("/api/:nom", async (req: Request, res: Response) => {
46   const pokemon = await recupererPokemon(req.params.nom)
47   if (!pokemon) {
48     return res.status(404).json({ erreur: "Pokemon introuvable dans la PokeAPI" })
49   }
50   res.json(pokemon)
51 })

```

fetch renvoie une Promise : on utilise await. On vérifie toujours `reponse.ok` (sinon un nom inexistant ferait planter le `.json()`). L'opérateur `?` et `??` gèrent les cas où une donnée manque.

7. Quête 7 — Capturer un Pokémon (API → base)

[300 XP]

C'est le cœur du sujet : on **récupère** depuis l'API, puis on **enregistre** dans Neon.

À faire

Ajoutez la route POST ci-dessous **dans le même fichier** `pokedex.routes.ts`.

Listing 7 – `pokedex.routes.ts` (partie 2 : capturer)

```

1 // POST : capturer un Pokemon -> /pokedex/capturer/pikachu
2 routerPokedex.post("/capturer/:nom", async (req: Request, res: Response) => {
3   const donnees = await recupererPokemon(req.params.nom)
4   if (!donnees) {
5     return res.status(404).json({ erreur: "Ce Pokemon n'existe pas dans la PokeAPI" })
6   }
7   try {
8     const pokemon = await prisma.pokemon.create({
9       data: donnees as any, // les types/enums viennent du mapping ci-dessus
10    })
11    res.status(201).json({ message: `${pokemon.nom} a ete capture !`, pokemon })
12  } catch (e) {
13    res.status(400).json({ erreur: "Deja capture (numeroPokedex unique) ?" })
14  }
15 })

```

Deux pièges :

- Les valeurs d'enum doivent correspondre **exactement** (en majuscules) à celles du schéma. C'est tout l'intérêt du dictionnaire `TYPES` : convertir `"fire"` → `"FEU"`. Une valeur inconnue

ferait échouer le `create`.

- Le `as any` sur `data` évite que l'éditeur râle parce que nos chaînes ne sont pas *typées* comme l'enum. À l'exécution (`tsx`), tout fonctionne tant que les valeurs sont correctes.

8. Quête 8 — Gérer son Pokédex

[200 XP]

À faire

Ajoutez les routes pour **consulter** le Pokédex (base) et **relâcher** un Pokémon.

Listing 8 – `pokedex.routes.ts` (partie 3 : la base)

```

1 // GET : mon Pokédex (Pokemon captures, depuis la base) -> /pokedex
2 routerPokedex.get("/", async (req: Request, res: Response) => {
3   const pokemons = await prisma.pokemon.findMany({
4     orderBy: { numeroPokedex: "asc" },
5     include: { dresseur: true },
6   })
7   res.json(pokemons)
8 })
9
10 // DELETE : relacher un Pokemon -> /pokedex/3
11 routerPokedex.delete("/:id", async (req: Request, res: Response) => {
12   const id = Number(req.params.id)
13   try {
14     const pokemon = await prisma.pokemon.delete({ where: { id } })
15     res.json({ message: `${pokemon.nom} a été relâché dans la nature !` })
16   } catch (e) {
17     res.status(404).json({ erreur: `Pokemon ${id} introuvable` })
18   }
19 })
20
21 export default routerPokedex

```

9. Quête 9 — Assembler le Centre (`server.ts`)

[150 XP]

Listing 9 – `src/server.ts`

```

1 import express from "express"
2 import dotenv from "dotenv"
3 import routerPokedex from "../routes/pokedex.routes"
4
5 dotenv.config()
6
7 const app = express()
8 app.use(express.json()) // parse le body JSON, avant les routes
9
10 app.use("/pokedex", routerPokedex)
11
12 const PORT = process.env.PORT || 3000
13 app.listen(PORT, () => {
14   console.log(`Serveur sur http://localhost:${PORT}`)
15 })

```

```
1 npm run dev
```

10. Quête 10 — Mettre à l'épreuve

[200 XP]

Dans le navigateur (GET)

URL	Résultat
localhost:3000/pokedex/api	la liste des Pokémon de l'API
localhost:3000/pokedex/api/pikachu	le détail de Pikachu (transformé)
localhost:3000/pokedex	votre Pokédex (vide au départ)

Avec un fichier `test.rest` (extension REST Client)

Listing 10 – test.rest

```
1  ### Voir les Pokemon disponibles dans l'API
2  GET http://localhost:3000/pokedex/api?limite=10
3
4  ### Voir le detail d'un Pokemon (API)
5  GET http://localhost:3000/pokedex/api/charizard
6
7  ### CAPTURER un Pokemon (le sauvegarde dans Neon)
8  POST http://localhost:3000/pokedex/capturer/charizard
9
10 ### Voir mon Pokedex (depuis la base)
11 GET http://localhost:3000/pokedex
12
13 ### Relacher le Pokemon d'id 1
14 DELETE http://localhost:3000/pokedex/1
```

Avec PowerShell

```
1  # Capturer Bulbizarre (bulbasaur)
2  Invoke-RestMethod -Method Post -Uri http://localhost:3000/pokedex/capturer/
   bulbasaur
3
4  # Consulter mon Pokedex
5  Invoke-RestMethod http://localhost:3000/pokedex | ConvertTo-Json -Depth 5
```

11. Quête 11 (Boss final) — Prouver la persistance

[200 XP]

Défi

1. Capturez 2 ou 3 Pokémon via POST `/pokedex/capturer/...`
2. **Arrêtez** le serveur (Ctrl+C) puis relancez `npm run dev`.
3. Refaites GET `/pokedex` : vos Pokémon sont toujours là → ils viennent de Neon.
4. Vérifiez dans `npx prisma studio` que les lignes existent dans la table `Pokemon`.

EXTENSION — Séance 5 : passer à Axios

Mise à jour du Pokédex ! Jusqu'ici on utilisait `fetch` (natif) pour interroger la PokéAPI. On va maintenant le remplacer par **Axios**, le client HTTP que la plupart des projets Node utilisent : code plus court, erreurs HTTP gérées automatiquement, et configuration centralisée. *Même comportement qu'avant, mais plus propre.*

12. Quête 12 — Installer Axios et créer une instance

[200 XP]

À faire

1. Installez Axios.
2. Créez une **instance** réutilisable avec l'URL de base de la PokéAPI.

```
1 npm install axios
```

Listing 11 – src/api/pokeapi.ts

```
1 import axios from "axios"
2
3 // Une seule instance, configurée une fois pour toutes
4 export const pokeapi = axios.create({
5   baseURL: "https://pokeapi.co/api/v2",
6   timeout: 5000, // abandonne après 5 secondes
7 })
```

C'est la même idée que le singleton `utils/prisma.ts` : on centralise la configuration (URL de base, timeout, en-têtes) en un seul endroit. Ailleurs, on écrira juste `pokeapi.get("/pokemon/pikachu")` au lieu de répéter l'URL complète. Axios fournit ses propres types : pas besoin de `@types/axios`.

13. Quête 13 — Refactoriser avec Axios

[300 XP]

À faire

Dans `pokedex.routes.ts`, remplacez `fetch` par l'instance `pokeapi`. Comparez avec votre version de la Quête 6.

Listing 12 – pokedex.routes.ts (version Axios)

```
1 import { Router, Request, Response } from "express"
2 import prisma from "../../utils/prisma"
3 import { pokeapi } from "../api/pokeapi"
4
5 const routerPokedex = Router()
6
7 const TYPES: Record<string, string> = {
8   normal: "NORMAL", fire: "FEU", water: "EAU", grass: "PLANTE",
9   electric: "ELECTRIK", ice: "GLACE", fighting: "COMBAT", poison: "POISON",
10  ground: "SOL", flying: "VOL", psychic: "PSY", bug: "INSECTE",
11  rock: "ROCHE", ghost: "SPECTRE", dragon: "DRAGON", dark: "TENEbres",
12  steel: "ACIER", fairy: "FEE",
13 }
14
```

```

15 // Recupere UN pokemon depuis la PokeAPI (avec Axios)
16 async function recupererPokemon(nom: string) {
17   try {
18     // Plus de .json() : les donnees sont DEJA dans data
19     const { data } = await pokeapi.get(`/pokemon/${nom.toLowerCase()}`)
20
21     const hp = data.stats.find((s: any) => s.stat.name === "hp").base_stat ?? 0
22
23     return {
24       numeroPokedex: data.id,
25       nom: data.name,
26       typePrincipal: TYPES[data.types[0].type.name] ?? "NORMAL",
27       typeSecondaire: data.types[1] ? TYPES[data.types[1].type.name] : null,
28       pointsVie: hp,
29       taille: data.height / 10,
30       poids: data.weight / 10,
31       imageUrl: data.sprites.front_default,
32     }
33   } catch (e) {
34     return null // 404 ou reseau : pokemon introuvable
35   }
36 }
37
38 // GET liste : on passe limite via params (Axios encode tout seul)
39 routerPokedex.get("/api", async (req: Request, res: Response) => {
40   const limite = Number(req.query.limite) || 20
41   const { data } = await pokeapi.get("/pokemon", { params: { limit: limite } })
42   res.json(data.results)
43 })

```

Ce qui disparaît avec Axios :

- Le double await : plus besoin de `await reponse.json()`, tout est dans `data`.
- La vérification `if (!reponse.ok)` : Axios lève une **erreur** sur un 404, donc c'est le `catch` qui s'en charge.
- La concaténation `?limit=...` : on passe un objet `params` et Axios construit la query string (et l'encode) pour nous.

Les routes `POST /capturer/:nom` et `GET /pokedex` ne changent pas : elles utilisent `recupererPokemon` et `Prisma` comme avant.

14. Quête 14 — Gérer les erreurs proprement

[200 XP]

Défi

Distinguez une **erreur HTTP** (le serveur a répondu 404/500) d'une **erreur réseau** (timeout, hors-ligne) avec `axios.isAxiosError`.

Listing 13 – Version détaillée du catch

```

1 async function recupererPokemon(nom: string) {
2   try {
3     const { data } = await pokeapi.get(`/pokemon/${nom.toLowerCase()}`)
4     // ... (transformation identique)
5     return data
6   } catch (e) {
7     if (axios.isAxiosError(e) && e.response) {
8       // le serveur a répondu (ex : 404 = pokemon inexistant)

```

```

9   console.log("Statut HTTP :", e.response.status)
10  } else {
11    // pas de reponse : timeout, DNS, hors-ligne...
12    console.log("Erreur reseau ou timeout")
13  }
14  return null
15  }
16  }

```

`e.response` existe *seulement* si le serveur a renvoyé une réponse. S'il n'existe pas, c'est un problème réseau (ou le `timeout: 5000` de l'instance s'est déclenché). `isAxiosError(e)` garantit à TypeScript que `e` est bien une erreur Axios avant d'accéder à `e.response`.

15. Quête 15 — Tous les verbes HTTP avec Axios

[200 XP]

Important : la PokéAPI est en **lecture seule** (on ne peut faire que des GET dessus). Pour pratiquer POST, PUT, PATCH et DELETE avec Axios, on s'adresse à **notre propre backend** (`http://localhost:3000`). C'est exactement ainsi qu'un frontend dialoguera avec votre API!

Axios a une méthode par verbe HTTP. Les voici en parallèle :

Verbe	Méthode Axios	Sens
GET	<code>axios.get(url, config)</code>	Lire
POST	<code>axios.post(url, corps, config)</code>	Créer
PUT	<code>axios.put(url, corps, config)</code>	Remplacer (tout)
PATCH	<code>axios.patch(url, corps, config)</code>	Modifier (en partie)
DELETE	<code>axios.delete(url, config)</code>	Supprimer

Pour `post`, `put` et `patch`, le **corps** (l'objet à envoyer) est le 2^e argument. Pour `get` et `delete`, il n'y a pas de corps : la config (`params`, `headers`...) est le 2^e argument.

À faire

Ajoutez une route PATCH à votre backend pour **modifier partiellement** un Pokémon (ex. le promouvoir au rang **LEGENDAIRE**).

Listing 14 – `pokedex.routes.ts` (ajout de PATCH)

```

1  // PATCH : modifier partiellement un Pokemon -> /pokedex/3
2  // body JSON, ex : { "rarete": "LEGENDAIRE" }
3  routerPokedex.patch("/:id", async (req: Request, res: Response) => {
4    const id = Number(req.params.id)
5    try {
6      const pokemon = await prisma.pokemon.update({
7        where: { id },
8        data: req.body,    // ne met a jour QUE les champs fournis
9      })
10     res.json(pokemon)
11   } catch (e) {
12     res.status(404).json({ erreur: 'Pokemon ${id} introuvable' })

```

```

13 }
14 })

```

PUT vs PATCH. PUT *remplace* la ressource entière (on envoie tous les champs), PATCH n'en *modifie qu'une partie*. Comme `prisma.update` ne touche que les champs présents dans `data`, il colle naturellement à la sémantique PATCH.

16. Quête 16 — Un client Axios qui pilote toute l'API

[300 XP]

Défi

Écrivez un petit script `src/client.ts` qui, avec Axios, enchaîne **POST** → **GET** → **PATCH** → **DELETE** sur votre propre backend. (Le serveur doit tourner dans un autre terminal.)

Listing 15 – `src/client.ts`

```

1 import axios from "axios"
2
3 // Une instance vers NOTRE backend
4 const api = axios.create({ baseUrl: "http://localhost:3000", timeout: 5000 })
5
6 async function demo() {
7   // 1) POST : capturer un Pokemon (les donnees viennent de la PokeAPI cote
8   // serveur)
9   const { data: capture } = await api.post("/pokedex/capturer/gengar")
10  const id = capture.pokemon.id
11  console.log("Capture :", capture.message)
12
13  // 2) GET : combien de Pokemon dans le Pokedex ?
14  const { data: pokedex } = await api.get("/pokedex")
15  console.log("Pokedex :", pokedex.length, "pokemon")
16
17  // 3) PATCH : promouvoir notre Pokemon au rang LEGENDAIRE
18  const { data: maj } = await api.patch(`/pokedex/${id}`, { rarete: "LEGENDAIRE" })
19  console.log("Nouvelle rarete :", maj.rarete)
20
21  // 4) DELETE : relacher le Pokemon
22  const { data: relache } = await api.delete(`/pokedex/${id}`)
23  console.log(relache.message)
24 }
25
26 demo().catch((e) => {
27   if (axios.isAxiosError(e)) {
28     console.log("Erreur API :", e.response?.status, e.response?.data)
29   } else {
30     console.log("Erreur :", e)
31   }
32 })

```

```

1 # Terminal 1 : le serveur
2 npm run dev
3
4 # Terminal 2 : le client
5 npx tsx src/client.ts

```

Vous venez d'utiliser Axios comme un **client** de votre propre API : c'est précisément le rôle qu'aura votre frontend (React, etc.). Notez l'usage de `e.response?.status` : le `?.` évite un plantage si l'erreur est réseau (donc sans `response`).

Quêtes bonus (XP supplémentaire)

- [+150 XP] Au moment de la capture, attribuez automatiquement la rareté **LEGENDAIRE** si `pointsVie` $\geq$ 100, sinon **COMMUN**.
- [+150 XP] Créez un **Dresseur** et associez chaque capture à son `dresseurId` (passez-le dans le *body* du POST).
- [+150 XP] Ajoutez GET `/pokedex?type=FEU` qui filtre les Pokémon capturés par `typePrincipal` (*where*).
- [+100 XP] Ajoutez GET `/pokedex/stats` qui renvoie le nombre de Pokémon capturés par `type` (*groupBy*).

Barème & rangs

XP total	Rang	
0 – 1000	Dresseur Débutant	Premier badge en vue
1001 – 1900	Dresseur Confirmé	4 badges
1901 – 2800	Champion d'Arène	8 badges
2801 +	Maître Pokémon	Vous les attraperez tous! ★

Attrapez-les tous... et que vos requêtes répondent toujours 200 OK !